Finales Handout

SciQ-Scorecards-Chatbot

Professionell formatierte Fassung für die Ausgabe an Studierende

---

Dokumentzweck

Dieses Handout dient als verbindliche Arbeitsgrundlage für die Umsetzung des SciQ-Scorecards-Chatbots. Es erläutert die fachliche Idee, die erforderlichen Implementierungsschritte, die relevanten Tensorformen sowie typische Fehlerquellen. Die Beschreibung ist auf eine erste praktische Umsetzung ausgerichtet.

Adressaten: Studierende der Lehrveranstaltung

Verwendungszweck: Begleitendes Handout zur Programmieraufgabe

Stand: 11. März 2026

Inhaltsstruktur

Das Dokument ist abschnittsweise aufgebaut. Jeder fachliche Abschnitt enthält eine präzise Erläuterung sowie einen eingerückten Pseudocode-Block. Die Reihenfolge entspricht der empfohlenen Implementierungsreihenfolge.

Nr.	Abschnitt
1	Überblick über die Aufgabe
2	Daten und Datensatzstruktur
3	Verwendete Modellgrundlage
4	Vorbereitung der Texte
5	Bildung von Textvektoren
6	Kontextvektor und Antwortvektoren
7	Merkmalsvektor pro Kontext-Antwort-Paar
8	Antwortpermutation und Label-Bestimmung
9	Dataset und Collate-Funktion
10	Erwartete Tensorformen
11	Aufbau des Modells
12	Training
13	Evaluation und Test
14	Nutzung von ask_bot
15	Drei automatische ask_bot-Aufrufe
16	Empfohlene Reihenfolge der Implementierung
17	Typische Fehlerquellen
18	Woran Sie eine korrekte Lösung erkennen
19	Schluss

1. Überblick über die Aufgabe

Sie bauen in dieser Aufgabe ein kleines Fragebeantwortungssystem mit vier Antwortoptionen. Der fachliche Fokus liegt nicht auf einem großen Sprachmodell, sondern auf einer sauberen Pipeline. Sie sollen zeigen, dass Sie aus festen vortrainierten Wortrepräsentationen, einer klaren Merkmalskonstruktion und einem kleinen neuronalen Netz ein funktionierendes System aufbauen können. Die Musterlösung verwendet dafür den SciQ-Datensatz, den Tokenizer von bert-base-uncased, die feste Eingabe-Embedding-Matrix dieses Modells, ein MLP zur Score-Berechnung und die Inferenzfunktion ask_bot.

Arbeiten Sie in der vorgegebenen Reihenfolge. Testen Sie jeden Zwischenschritt, bevor Sie zum nächsten übergehen. Beginnen Sie nicht mit dem Training, bevor Sie ein einzelnes Beispiel vollständig bis zu vier Logits durchrechnen können.

Pseudocode

Ablaufskizze

```
Ziel:
    Erzeuge ein System, das zu einer Frage aus vier Optionen die beste auswählt.

Reihenfolge:
    1. Datensatz verstehen
    2. Tokenisierung testen
    3. Embeddings laden
    4. Textvektoren bilden
    5. Merkmalsvektoren bauen
    6. Dataset und Collate-Funktion schreiben
    7. MLP implementieren
    8. Training und Evaluation ergänzen
    9. ask_bot implementieren
   10. ask_bot drei Mal automatisch aufrufen
```

2. Daten und Datensatzstruktur

Der Datensatz allenai/sciq enthält die Splits train, validation und test. Jedes Beispiel enthält eine Frage, einen optionalen Support-Text, die korrekte Antwort und drei Distraktoren. Für Ihre Implementierung ist entscheidend, dass die Antworten anfangs nur als Texte vorliegen. Das numerische Label entsteht erst später, nachdem Sie die Antwortreihenfolge festgelegt oder permutiert haben.

Sehen Sie sich mehrere Beispiele zu Beginn an. Prüfen Sie dabei, wie Fragen, Support-Texte und Antworten typischerweise aussehen. Dieser Schritt ist notwendig, weil Sie nur dann sinnvolle Token-Grenzen und eine korrekte Datenverarbeitung festlegen können.

Pseudocode

Ablaufskizze

```
Lade Datensatz
    dataset = load_dataset("allenai/sciq")

Greife auf Splits zu
    train_split = dataset["train"]
    val_split   = dataset["validation"]
    test_split  = dataset["test"]

Untersuche Beispiel
    example = train_split[0]
    frage    = example["question"]
    support  = example["support"]
    korrekt  = example["correct_answer"]
    falsch_1 = example["distractor1"]
    falsch_2 = example["distractor2"]
    falsch_3 = example["distractor3"]
```

3. Verwendete Modellgrundlage

Sie verwenden den Tokenizer von bert-base-uncased und extrahieren aus AutoModel.from_pretrained("bert-base-uncased") ausschließlich die Eingabe-Embeddings. Die Musterlösung speichert diese Embedding-Matrix als feste Matrix und nutzt sie danach direkt für die Mittelwertsvektoren. Der eigentliche BERT-Encoder wird nicht verwendet. Genau das ist eine zentrale fachliche Vorgabe dieser Aufgabe.

Sie dürfen daher insbesondere nicht mit model(**inputs) arbeiten, keine Hidden States weiterverarbeiten und keinen Pooler verwenden. Trainierbar ist nur Ihr MLP.

Pseudocode

Ablaufskizze

```
Lade Tokenizer
    tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

Lade Modell nur für Embeddings
    hf_model = AutoModel.from_pretrained("bert-base-uncased")

Extrahiere Input-Embeddings
    emb_matrix = hf_model.get_input_embeddings().weight

Verwende danach nur:
    Token-ID -> Embedding-Vektor

Verwende nicht:
    hf_model(**inputs)
    last_hidden_state
    pooler_output
    attentions
```

4. Vorbereitung der Texte

Tokenisieren Sie Frage, Support und Antworten ohne Special Tokens. Eine sinnvolle Standardkonfiguration ist eine maximale Länge von 48 Tokens für Fragen, 96 Tokens für Support und 28 Tokens für Antworten. Die Musterlösung verwendet genau diese Größen. Außerdem behandelt sie leere Texte robust: None oder leere Strings werden zu leeren Tokenlisten, und eine leere Tokenliste wird später zu einem Nullvektor.

Dieser Punkt ist operativ wichtig. Wenn Sie bei einer leeren Tokenliste dennoch einen Mittelwert berechnen, erzeugen Sie undefinierte Werte. Das führt typischerweise zu NaNs und instabilem Training.

Pseudocode

Ablaufskizze

```
Definiere Grenzen
    MAX_Q = 48
    MAX_S = 96
    MAX_A = 28

Funktion tokenize_text(text, max_len):
    falls text None ist:
        return leere Liste
```

```
    falls text leer ist:
        return leere Liste

    ids = tokenizer.encode(
        text,
        add_special_tokens=False,
        truncation=True,
        max_length=max_len
    )
    return ids
```

5. Bildung von Textvektoren

Eine Tokenliste wird mithilfe der festen Embedding-Matrix in eine Folge von Token-Embeddings umgewandelt. Anschließend bilden Sie über alle Token den Mittelwert. Das Ergebnis ist ein einzelner Textvektor. Bei bert-base-uncased hat dieser Vektor die Länge 768. Für leere Tokenlisten geben Sie stattdessen einen Nullvektor dieser Länge zurück. Genau so verfährt die Musterlösung.

Sie bauen hier bewusst kein Sequenzmodell. Die Aufgabe verlangt keine Rekurrenz und keine Selbstaufmerksamkeit. Der Mittelwert der Token-Embeddings ist die vorgesehene Repräsentation.

Pseudocode

Ablaufskizze

```
Funktion mean_embedding(token_ids, emb_matrix):
    falls token_ids leer:
        return Nullvektor der Länge embedding_dim

    hole für jede Token-ID den Embedding-Vektor
    bilde den Mittelwert über alle Token
    return Textvektor
```

6. Kontextvektor und Antwortvektoren

Der Kontext eines Beispiels besteht aus Frage und Support zusammen. Die Musterlösung tokenisiert beide Texte getrennt und hängt danach die Tokenlisten aneinander. Aus der zusammengeführten Liste wird dann der Kontextvektor berechnet. Für jede der vier Antwortoptionen wird separat ein Antwortvektor gebildet. Alle vier Antwortoptionen werden also gegen denselben Kontext verglichen.

Das ist wichtig für die spätere Logik des Modells. Sie berechnen nicht einen gemeinsamen Vektor für alle Antworten, sondern genau vier Kontext-Antwort-Paare pro Beispiel.

Pseudocode

Ablaufskizze

```
frage_ids   = tokenize_text(frage, MAX_Q)
support_ids = tokenize_text(support, MAX_S)

kontext_ids = frage_ids + support_ids
kontext_vec = mean_embedding(kontext_ids, emb_matrix)

für jede Antwort in den vier Optionen:
    antwort_ids = tokenize_text(antwort, MAX_A)
    antwort_vec = mean_embedding(antwort_ids, emb_matrix)
```

7. Merkmalsvektor pro Kontext-Antwort-Paar

Für jedes Paar aus Kontextvektor und Antwortvektor bauen Sie einen Merkmalsvektor aus vier Blöcken auf: Kontext, Antwort, absolute Differenz und elementweises Produkt. Die Musterlösung verwendet genau diese Konstruktion. Wenn die Embedding-Dimension 768 beträgt, hat jeder Block 768 Werte und der vollständige Merkmalsvektor damit 3072 Werte.

Diese Konstruktion ist fachlich sinnvoll, weil sie dem MLP sowohl die beiden Ausgangsvektoren als auch direkte Vergleichsinformation liefert.

Pseudocode

Ablaufskizze

```
Funktion build_feature_vector(kontext_vec, antwort_vec):
    diff_vec = abs(kontext_vec - antwort_vec)
    prod_vec = kontext_vec * antwort_vec

    feature_vec = concat(
        kontext_vec,
        antwort_vec,
        diff_vec,
        prod_vec
    )

    return feature_vec
```

8. Antwortpermutation und Label-Bestimmung

Die Ausgangsreihenfolge der Antworten lautet immer: korrekte Antwort, Distraktor 1, Distraktor 2, Distraktor 3. Danach werden die vier Antworten permutiert. Das Label ist anschließend nicht mehr automatisch 0, sondern die neue Position der ursprünglich korrekten Antwort. In der Musterlösung ist diese Logik in __getitem__ umgesetzt. Für Training wird zufällig permutiert, für Evaluation deterministisch mit einem aus Seed und Index erzeugten Zufallszahlengenerator.

Dieser Schritt ist eine der häufigsten Fehlerquellen. Wenn Sie die Permutation korrekt anwenden, das Label aber nicht anpassen, trainiert das Modell mit falschen Zielwerten.

Pseudocode

Ablaufskizze

```
antworten = [korrekt, distr1, distr2, distr3]

falls Split = train:
    perm = zufällige Permutation von [0, 1, 2, 3]
sonst:
    perm = reproduzierbare Permutation mit seed + index

permutierte_antworten = [antworten[i] für i in perm]

label = Position der ursprünglichen Antwort 0 in perm
```

9. Dataset und Collate-Funktion

Das Dataset liefert pro Beispiel die vorbereiteten Tokenlisten für Frage, Support und die vier permutierten Antworten sowie das berechnete Label. Die Collate-Funktion übernimmt anschließend die Batch-Bildung. Sie konstruiert für jede Antwortoption den Merkmalsvektor und erzeugt daraus einen Tensor der Form (B, 4, F). Die Labels haben die Form (B,). Genau diese Tensorformen nutzt die Musterlösung.

Sie benötigen in dieser Aufgabe kein Padding der Tokenlisten, weil Sie jede Tokenliste vor der Batch-Bildung in einen festen Vektor überführen. Das vereinfacht die Implementierung deutlich.

Pseudocode

Ablaufskizze

```
Dataset __getitem__(index):
    lese Rohtexte
    baue Antwortliste [korrekt, distr1, distr2, distr3]
    permutiere Antworten
    bestimme Label
    tokenisiere Frage, Support und Antworten
    return Beispielstruktur

Collate-Funktion(batch):
    für jedes Beispiel im Batch:
        berechne Kontextvektor
        berechne vier Antwortvektoren
        baue vier Merkmalsvektoren
        stacke zu Form (4, F)

    stacke alle Beispiele zu Form (B, 4, F)
    baue Label-Tensor der Form (B,)
    return X, y
```

10. Erwartete Tensorformen

Die wichtigsten Tensorformen müssen Ihnen in jedem Schritt klar sein. Eine Tokenliste hat Form (T,). Die Token-Embedding-Folge eines Textes hat Form (T, 768). Ein Textvektor hat Form (768,). Ein Merkmalsvektor hat Form (3072,). Ein Beispiel mit vier Antworten hat Form (4, 3072). Ein Batch hat Form (B, 4, 3072). Die Logits des Modells haben Form (B, 4). Die Musterlösung folgt genau diesem Schema.

Wenn Ihr Code an einer Stelle andere Formen erzeugt, sollten Sie den Fehler sofort beheben. Warten Sie in diesem Fall nicht bis zum Training.

Pseudocode

Ablaufskizze

```
Kontrolliere Formen
    token_ids        -> (T,)
    token_embs       -> (T, 768)
    text_vec         -> (768,)
    feature_vec      -> (3072,)
    example_features -> (4, 3072)
    batch_features   -> (B, 4, 3072)
    logits           -> (B, 4)
    labels           -> (B,)
```

11. Aufbau des Modells

Das Modell ist ein kleines MLP mit einer linearen Schicht, ReLU, Dropout und einer Ausgabeschicht, die pro Antwortoption genau einen Score liefert. Die Musterlösung verwendet diese Architektur mit hidden_dim = 512 und dropout = 0.30. Im Forward-Schritt wird der Tensor von (B, 4, F) auf (B*4, F) umgeformt, gemeinsam durch das MLP geführt und danach wieder zu (B, 4) zurückgeformt.

Wichtig ist außerdem: Vor CrossEntropyLoss wird keine Softmax angewendet. Die Verlustfunktion erwartet rohe Logits.

Pseudocode

Ablaufskizze

```
Klasse ScorecardsMLP:
    fc1  : Linear(F, H)
    relu : ReLU()
    drop : Dropout(p)
    fc2  : Linear(H, 1)

Forward(X):
    X hat Form (B, 4, F)
    forme um zu (B*4, F)
    h = fc1(X)
    h = relu(h)
    h = drop(h)
    s = fc2(h)
    forme um zu (B, 4)
    return logits
```

12. Training

Die Musterlösung verwendet AdamW, CrossEntropyLoss, BATCH_SIZE = 16, LEARNING_RATE = 3e-4, WEIGHT_DECAY = 1e-2 und EPOCHS = 30. Diese Werte können Sie direkt übernehmen. In jeder Epoche wird trainiert, anschließend auf dem Validierungssplit evaluiert und bei verbesserter Validation Accuracy der Modellzustand gespeichert.

Die Standardreihenfolge pro Batch lautet: Daten auf das Zielgerät verschieben, Gradienten nullen, Logits berechnen, Verlust berechnen, backward(), optimizer.step().

Pseudocode

Ablaufskizze

```
Initialisiere
    model = ScorecardsMLP(...)
    optimizer = AdamW(...)
    loss_fn = CrossEntropyLoss()

Für jede Epoche:
    setze Modell in Trainingsmodus

    für X, y in train_loader:
        verschiebe X und y auf Gerät
        optimizer.zero_grad()
        logits = model(X)
```

```
        loss = loss_fn(logits, y)
        loss.backward()
        optimizer.step()

    berechne Validation Accuracy
    speichere Modell, falls Validation Accuracy besser ist
```

13. Evaluation und Test

Evaluation bedeutet hier: kein Training, keine Gradienten, nur Vorwärtsrechnung und Accuracy-Berechnung. Die Musterlösung verwendet dafür eine eigene Funktion evaluate. Nach Abschluss des Trainings wird der beste Modellzustand geladen und anschließend die Test Accuracy berechnet. Der Testsplit dient nur der abschließenden Bewertung, nicht der Modellauswahl.

Diese Trennung ist verbindlich, weil nur so eine saubere Leistungsbewertung möglich ist.

Pseudocode

Ablaufskizze

```
Funktion evaluate(model, loader):
    setze Modell in Evaluationsmodus
    korrekt = 0
    gesamt  = 0

    ohne Gradienten:
        für X, y in loader:
            logits = model(X)
            pred = argmax(logits, Achse=1)
            erhöhe korrekt und gesamt

    return korrekt / gesamt

Nach Training:
    lade bestes Modell
    test_acc = evaluate(model, test_loader)
```

14. Nutzung von ask_bot

Die Funktion ask_bot(question, options, support="") verwendet dieselbe Pipeline wie das trainierte System auf dem Datensatz. Sie prüft, ob genau vier Optionen übergeben wurden, berechnet daraus vier Merkmalsvektoren, ruft das Modell auf, berechnet Wahrscheinlichkeiten über Softmax und gibt die vom Modell gewählte Antwort aus. In der Musterlösung werden zusätzlich die Wahrscheinlichkeiten aller vier Optionen ausgegeben.

Im Unterschied zum Training werden die vier übergebenen Antwortoptionen in ask_bot nicht permutiert. Die Reihenfolge der Benutzereingabe bleibt erhalten.

Pseudocode

Ablaufskizze

```
Funktion ask_bot(question, options, support=""):
    prüfe, dass genau vier Optionen vorhanden sind

    tokenisiere Frage und Support
    berechne Kontextvektor
```

```
    für jede Option:
        tokenisiere Antwort
        berechne Antwortvektor
        baue Merkmalsvektor

    stacke zu Form (1, 4, F)
    setze Modell in Evaluationsmodus
    berechne logits
    berechne Wahrscheinlichkeiten
    best_index = argmax(Wahrscheinlichkeiten)

    gib gewählte Option und Wahrscheinlichkeiten aus
```

15. Drei automatische ask_bot-Aufrufe

Die Musterlösung beendet das Programm nicht mit der Definition von ask_bot, sondern ruft die Funktion anschließend drei Mal automatisch auf. Dabei werden drei Beispiele aus dem Testdatensatz verwendet. Vor jedem Aufruf wird zusätzlich die erwartete korrekte Antwort ausgegeben. Dieser Schritt gehört zur funktionalen Vollständigkeit des Programms und soll sicherstellen, dass die Inferenzfunktion sichtbar demonstriert wird.

Wenn Sie ask_bot nur definieren, aber nie aufrufen, ist die Anwendung aus Sicht der Aufgabenstellung nicht vollständig demonstriert.

Pseudocode

Ablaufskizze

```
Funktion run_three_ask_bot_examples(test_dataset):
    n = Minimum aus 3 und Länge des Testdatensatzes

    für i von 0 bis n-1:
        ex = test_dataset[i]
        gib erwartete korrekte Antwort aus

        ask_bot(
            question = ex["question"],
            options  = ex["answer_texts"],
            support  = ex["support"]
        )

Am Programmende:
    run_three_ask_bot_examples(test_dataset)
```

16. Empfohlene Reihenfolge der Implementierung

Implementieren Sie die Lösung nicht in einem Schritt. Gehen Sie in dieser Reihenfolge vor: Datensatz laden, Beispiele prüfen, Tokenisierung testen, Embedding-Matrix laden, Mittelwertsvektor berechnen, Merkmalsvektor berechnen, Dataset schreiben, Collate-Funktion schreiben, Modell testen, Training ergänzen, Evaluation ergänzen, ask_bot ergänzen, automatische drei Demo-Aufrufe ergänzen. Diese Reihenfolge entspricht der Logik der Musterlösung und reduziert Fehlersuche erheblich.

Pseudocode

Ablaufskizze

```
Implementationsplan
    1. dataset laden
    2. Beispiel inspizieren
    3. tokenize_text testen
    4. mean_embedding testen
    5. build_feature_vector testen
    6. Dataset implementieren
    7. Collate-Funktion implementieren
    8. Modell auf einem Batch testen
    9. Training ergänzen
   10. Evaluation ergänzen
   11. ask_bot ergänzen
   12. drei automatische ask_bot-Aufrufe ergänzen
```

17. Typische Fehlerquellen

Die häufigsten Fehler in dieser Aufgabe sind klar identifizierbar. Erstens wird versehentlich der vollständige BERT-Encoder verwendet. Zweitens wird der Support-Text ignoriert. Drittens wird das Label nach der Permutation nicht neu bestimmt. Viertens führen leere Texte zu undefinierten Mittelwerten. Fünftens werden Tensorformen verwechselt. Sechstens wird vor CrossEntropyLoss fälschlich Softmax angewendet. Siebtens wird ask_bot definiert, aber nicht demonstriert. Alle diese Fehler widersprechen der Musterlösung oder führen zu einem faktisch unbrauchbaren Ergebnis.

Pseudocode

Ablaufskizze

```
Fehlerprüfung
    prüfe: wird nur die Embedding-Matrix verwendet?
    prüfe: geht Support in den Kontext ein?
    prüfe: stimmt das Label nach Permutation?
    prüfe: erzeugen leere Texte Nullvektoren?
    prüfe: hat X Form (B, 4, F)?
    prüfe: haben logits Form (B, 4)?
    prüfe: keine Softmax vor CrossEntropyLoss
    prüfe: ask_bot wird am Ende aufgerufen
```

18. Woran Sie eine korrekte Lösung erkennen

Ihre Lösung ist fachlich vollständig, wenn das Programm den Datensatz korrekt lädt, alle relevanten Texttypen tokenisiert, feste Embeddings verwendet, Batch-Tensoren in der richtigen Form erzeugt, ein MLP trainiert, das beste Modell anhand der Validation Accuracy auswählt, die Test Accuracy berechnet und ask_bot anschließend drei Mal automatisch ausführt. Genau diese Struktur ist in der Musterlösung umgesetzt.

Eine niedrige Accuracy ist zunächst kein Grund, die Architektur zu ändern. Prüfen Sie zuerst Datenvorbereitung, Permutation, Label, Tensorformen und Support-Verarbeitung.

Pseudocode

Ablaufskizze

```
Abschlusskontrolle
```

```
    läuft das Programm ohne manuellen Eingriff?
    werden Validation und Test berechnet?
    wird das beste Modell geladen?
    funktioniert ask_bot mit vier Optionen?
    erfolgen drei automatische Demo-Aufrufe?
    sind alle zentralen Formen korrekt?
```

19. Schluss

Lesen Sie dieses Handout als direkte Implementierungsanleitung. Die Aufgabe ist bewusst so definiert, dass Sie keine zusätzliche Modellkomplexität erfinden müssen. Entscheidend ist eine saubere, konsequent umgesetzte Pipeline. Wenn Sie die Schritte in der angegebenen Reihenfolge bearbeiten und jeden Zwischenschritt prüfen, können Sie die vollständige Lösung zuverlässig umsetzen.